

```

44     delete ref;
45     free(ref_count);
46     ref = NULL;
47     ref_count = NULL;
48 }
49 }
50
51 T * ref;
52 unsigned * ref_count;
53 };

```

The code for this problem is complicated, and you probably wouldn't be expected to complete it flawlessly.

12.10 Malloc: Write an aligned malloc and free function that supports allocating memory such that the memory address returned is divisible by a specific power of two.

EXAMPLE

`align_malloc(1000,128)` will return a memory address that is a multiple of 128 and that points to memory of size 1000 bytes.

`aligned_free()` will free memory allocated by `align_malloc`.

pg 164

SOLUTION

Typically, with `malloc`, we do not have control over where the memory is allocated within the heap. We just get a pointer to a block of memory which could start at any memory address within the heap.

We need to work with these constraints by requesting enough memory that we can return a memory address which is divisible by the desired value.

Suppose we are requesting a 100-byte chunk of memory, and we want it to start at a memory address that is a multiple of 16. How much extra memory would we need to allocate to ensure that we can do so? We would need to allocate an extra 15 bytes. With these 15 bytes, plus another 100 bytes right after that sequence, we know that we would have a memory address divisible by 16 with space for 100 bytes.

We could then do something like:

```

1 void* aligned_malloc(size_t required_bytes, size_t alignment) {
2     int offset = alignment - 1;
3     void* p = (void*) malloc(required_bytes + offset);
4     void* q = (void*) (((size_t)(p) + offset) & ~(alignment - 1));
5     return q;
6 }

```

Line 4 is a bit tricky, so let's discuss it. Suppose `alignment` is 16. We know that one of the first 16 memory address in the block at `p` must be divisible by 16. With `(p + 15) & 11...10000` we advance as need to this address. ANDing the last four bits of `p + 15` with `0000` guarantees that this new value will be divisible by 16 (either at the original `p` or in one of the following 15 addresses).

This solution is *almost* perfect, except for one big issue: how do we free the memory?

We've allocated an extra 15 bytes, in the above example, and we need to free them when we free the "real" memory.

We can do this by storing, in this "extra" memory, the address of where the full memory block begins. We will store this immediately before the aligned memory block. Of course, this means that we now need to allocate even *more* extra memory to ensure that we have enough space to store this pointer.

Therefore, to guarantee both an aligned address and space for this pointer, we will need to allocate an additional $\text{alignment} - 1 + \text{sizeof}(\text{void}^*)$ bytes.

The code below implements this approach.

```

1 void* aligned_malloc(size_t required_bytes, size_t alignment) {
2     void* p1; // initial block
3     void* p2; // aligned block inside initial block
4     int offset = alignment - 1 + sizeof(void*);
5     if ((p1 = (void*)malloc(required_bytes + offset)) == NULL) {
6         return NULL;
7     }
8     p2 = (void*)((size_t)(p1) + offset) & ~(alignment - 1);
9     ((void**)p2)[-1] = p1;
10    return p2;
11 }
12
13 void aligned_free(void *p2) {
14     /* for consistency, we use the same names as aligned_malloc */
15     void* p1 = ((void**)p2)[-1];
16     free(p1);
17 }

```

Let's look at the pointer arithmetic in lines 9 and 15. If we treat `p2` as a `void**` (or an array of `void*`'s), we can just look at the index `- 1` to retrieve `p1`.

In `aligned_free`, we take `p2` as the same `p2` returned from `aligned_malloc`. As before, we know that the value of `p1` (which points to the beginning of the full memory block) was stored just before `p2`. By freeing `p1`, we deallocate the whole memory block.

12.11 2D Alloc: Write a function in C called `my2DAlloc` which allocates a two-dimensional array. Minimize the number of calls to `malloc` and make sure that the memory is accessible by the notation `arr[i][j]`.

pg 164

SOLUTION

As you may know, a two-dimensional array is essentially an array of arrays. Since we use pointers with arrays, we can use double pointers to create a double array.

The basic idea is to create a one-dimensional array of pointers. Then, for each array index, we create a new one-dimensional array. This gives us a two-dimensional array that can be accessed via array indices.

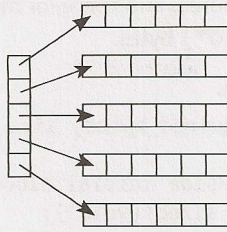
The code below implements this.

```

1 int** my2DAlloc(int rows, int cols) {
2     int** rowptr;
3     int i;
4     rowptr = (int**) malloc(rows * sizeof(int*));
5     for (i = 0; i < rows; i++) {
6         rowptr[i] = (int*) malloc(cols * sizeof(int));
7     }
8     return rowptr;
9 }

```

Observe how, in the above code, we've told `rowptr` where exactly each index should point. The following diagram represents how this memory is allocated.



To free this memory, we cannot simply call `free` on `rowptr`. We need to make sure to free not only the memory from the first `malloc` call, but also each subsequent call.

```
1 void my2DDealloc(int** rowptr, int rows) {
2     for (i = 0; i < rows; i++) {
3         free(rowptr[i]);
4     }
5     free(rowptr);
6 }
```

Rather than allocating the memory in many different blocks (one block for each row, plus one block to specify *where* each row is located), we can allocate this in a consecutive block of memory. Conceptually, for a two-dimensional array with five rows and six columns, this would look like the following.



If it seems strange to view the 2D array like this (and it probably does), remember that this is fundamentally no different than the first diagram. The only difference is that the memory is in a contiguous block, so our first five (in this example) elements point elsewhere in the same block of memory.

To implement this solution, we do the following.

```
1 int** my2DAlloc(int rows, int cols) {
2     int i;
3     int header = rows * sizeof(int*);
4     int data = rows * cols * sizeof(int);
5     int** rowptr = (int**)malloc(header + data);
6     if (rowptr == NULL) return NULL;
7
8     int* buf = (int*) (rowptr + rows);
9     for (i = 0; i < rows; i++) {
10         rowptr[i] = buf + i * cols;
11     }
12     return rowptr;
13 }
```

You should carefully observe what is happening on lines 11 through 13. If there are five rows of six columns each, `array[0]` will point to `array[5]`, `array[1]` will point to `array[11]`, and so on.

Then, when we actually call `array[1][3]`, the computer looks up `array[1]`, which is a pointer to another spot in memory—specifically, a pointer to `array[5]`. This element is treated as its own array, and we then get the third (zero-indexed) element from it.

Constructing the array in a single call to `malloc` has the added benefit of allowing disposal of the array with a single `free` call rather than using a special function to free the remaining data blocks.

13

Solutions to Java

13.1 Private Constructor: In terms of inheritance, what is the effect of keeping a constructor private?

pg 167

SOLUTION

Declaring a constructor `private` on class A means that you can only access the (private) constructor if you could also access A's private methods. Who, other than A, can access A's private methods and constructor? A's inner classes can. Additionally, if A is an inner class of Q, then Q's other inner classes can.

This has direct implications for inheritance, since a subclass calls its parent's constructor. The class A can be inherited, but only by its own or its parent's inner classes.

13.2 Return from Finally: In Java, does the finally block get executed if we insert a return statement inside the try block of a try-catch-finally?

pg 167

SOLUTION

Yes, it will get executed. The `finally` block gets executed when the `try` block exits. Even when we attempt to exit within the `try` block (via a `return` statement, a `continue` statement, a `break` statement or any exception), the `finally` block will still be executed.

Note that there are some cases in which the `finally` block will not get executed, such as the following:

- If the virtual machine exits during `try/catch` block execution.
- If the thread which is executing during the `try/catch` block gets killed.

13.3 Final, etc.: What is the difference between `final`, `finally`, and `finalize`?

pg 167

SOLUTIONS

Despite their similar sounding names, `final`, `finally` and `finalize` have very different purposes. To speak in very general terms, `final` is used to control whether a variable, method, or class is "changeable." The `finally` keyword is used in a `try/catch` block to ensure that a segment of code is always executed. The `finalize()` method is called by the garbage collector once it determines that no more references exist.

Further detail on these keywords and methods is provided below.

final

The final statement has a different meaning depending on its context.

- When applied to a variable (primitive): The value of the variable cannot change.
- When applied to a variable (reference): The reference variable cannot point to any other object on the heap.
- When applied to a method: The method cannot be overridden.
- When applied to a class: The class cannot be subclassed.

finally keyword

There is an optional finally block after the try block or after the catch block. Statements in the finally block will always be executed, even if an exception is thrown (except if Java Virtual Machine exits from the try block). The finally block is often used to write the clean-up code. It will be executed after the try and catch blocks, but before control transfers back to its origin.

Watch how this plays out in the example below.

```
1 public static String lem() {
2     System.out.println("lem");
3     return "return from lem";
4 }
5
6 public static String foo() {
7     int x = 0;
8     int y = 5;
9     try {
10        System.out.println("start try");
11        int b = y / x;
12        System.out.println("end try");
13        return "returned from try";
14    } catch (Exception ex) {
15        System.out.println("catch");
16        return lem() + " | returned from catch";
17    } finally {
18        System.out.println("finally");
19    }
20 }
21
22 public static void bar() {
23     System.out.println("start bar");
24     String v = foo();
25     System.out.println(v);
26     System.out.println("end bar");
27 }
28
29 public static void main(String[] args) {
30     bar();
31 }
```

The output for this code is the following:

```
1 start bar
```



```

2  start try
3  catch
4  lem
5  finally
6  return from lem | returned from catch
7  end bar

```

Look carefully at lines 3 to 5 in the output. The `catch` block is fully executed (including the function call in the `return` statement), then the `finally` block, and then the function actually returns.

finalize()

The automatic garbage collector calls the `finalize()` method just before actually destroying the object. A class can therefore override the `finalize()` method from the `Object` class in order to define custom behavior during garbage collection.

```

1  protected void finalize() throws Throwable {
2      /* Close open files, release resources, etc */
3  }

```

13.4 Generics vs. Templates: Explain the difference between templates in C++ and generics in Java.

pg 167

SOLUTION

Many programmers consider templates and generics to be essentially equivalent because both allow you to do something like `List<String>`. But, *how* each language does this, and *why*, varies significantly.

The implementation of Java generics is rooted in an idea of “type erasure.” This technique eliminates the parameterized types when source code is translated to the Java Virtual Machine (JVM) byte code.

For example, suppose you have the Java code below:

```

1  Vector<String> vector = new Vector<String>();
2  vector.add(new String("hello"));
3  String str = vector.get(0);

```

During compilation, this code is re-written into:

```

1  Vector vector = new Vector();
2  vector.add(new String("hello"));
3  String str = (String) vector.get(0);

```

The use of Java generics didn’t really change much about our capabilities; it just made things a bit prettier. For this reason, Java generics are sometimes called “syntactic sugar.”

This is quite different from C++. In C++, templates are essentially a glorified macro set, with the compiler creating a new copy of the template code for each type. Proof of this is in the fact that an instance of `MyClass<Foo>` will not share a static variable with `MyClass<Bar>`. Two instances of `MyClass<Foo>`, however, will share a static variable.

To illustrate this, consider the code below:

```

1  /** MyClass.h */
2  template<class T> class MyClass {
3  public:
4      static int val;
5      MyClass(int v) { val = v; }
6  };
7

```

```

8  /*** MyClass.cpp ***/
9  template<typename T>
10 int MyClass<T>::bar;
11
12 template class MyClass<Foo>;
13 template class MyClass<Bar>;
14
15 /*** main.cpp ***/
16 MyClass<Foo> * foo1 = new MyClass<Foo>(10);
17 MyClass<Foo> * foo2 = new MyClass<Foo>(15);
18 MyClass<Bar> * bar1 = new MyClass<Bar>(20);
19 MyClass<Bar> * bar2 = new MyClass<Bar>(35);
20
21 int f1 = foo1->val; // will equal 15
22 int f2 = foo2->val; // will equal 15
23 int b1 = bar1->val; // will equal 35
24 int b2 = bar2->val; // will equal 35

```

In Java, static variables are shared across instances of `MyClass`, regardless of the different type parameters.

Java generics and C++ templates have a number of other differences. These include:

- C++ templates can use primitive types, like `int`. Java cannot and must instead use `Integer`.
- In Java, you can restrict the template's type parameters to be of a certain type. For instance, you might use generics to implement a `CardDeck` and specify that the type parameter must extend from `CardGame`.
- In C++, the type parameter can be instantiated, whereas Java does not support this.
- In Java, the type parameter (i.e., the `Foo` in `MyClass<Foo>`) cannot be used for static methods and variables, since these would be shared between `MyClass<Foo>` and `MyClass<Bar>`. In C++, these classes are different, so the type parameter can be used for static methods and variables.
- In Java, all instances of `MyClass`, regardless of their type parameters, are the same type. The type parameters are erased at runtime. In C++, instances with different type parameters are different types.

Remember: Although Java generics and C++ templates look the same in many ways, they are very different.

13.5 TreeMap, HashMap, LinkedHashMap: Explain the differences between `TreeMap`, `HashMap`, and `LinkedHashMap`. Provide an example of when each one would be best.

pg 167

SOLUTION

All offer a key->value map and a way to iterate through the keys. The most important distinction between these classes is the time guarantees and the ordering of the keys.

- `HashMap` offers $O(1)$ lookup and insertion. If you iterate through the keys, though, the ordering of the keys is essentially arbitrary. It is implemented by an array of linked lists.
- `TreeMap` offers $O(\log N)$ lookup and insertion. Keys are ordered, so if you need to iterate through the keys in sorted order, you can. This means that keys must implement the `Comparable` interface. `TreeMap` is implemented by a Red-Black Tree.
- `LinkedHashMap` offers $O(1)$ lookup and insertion. Keys are ordered by their insertion order. It is implemented by doubly-linked buckets.

Imagine you passed an empty `TreeMap`, `HashMap`, and `LinkedHashMap` into the following function:

```

1 void insertAndPrint(AbstractMap<Integer, String> map) {
2     int[] array = {1, -1, 0};
3     for (int x : array) {
4         map.put(x, Integer.toString(x));
5     }
6
7     for (int k : map.keySet()) {
8         System.out.print(k + ", ");
9     }
10 }

```

The output for each will look like the results below.

HashMap	LinkedHashMap	TreeMap
(any ordering)	{1, -1, 0}	{-1, 0, 1}

Very important: The output of `LinkedHashMap` and `TreeMap` must look like the above. For `HashMap`, the output was, in my own tests, `{0, 1, -1}`, but it could be any ordering. There is no *guarantee* on the ordering.

When might you need ordering in real life?

- Suppose you were creating a mapping of names to `Person` objects. You might want to periodically output the people in alphabetical order by name. A `TreeMap` lets you do this.
- A `TreeMap` also offers a way to, given a name, output the next 10 people. This could be useful for a “More” function in many applications.
- A `LinkedHashMap` is useful whenever you need the ordering of keys to match the ordering of insertion. This might be useful in a caching situation, when you want to delete the oldest item.

Generally, unless there is a reason not to, you would use `HashMap`. That is, if you need to get the keys back in insertion order, then use `LinkedHashMap`. If you need to get the keys back in their true/natural order, then use `TreeMap`. Otherwise, `HashMap` is probably best. It is typically faster and requires less overhead.

13.6 Object Reflection: Explain what object reflection is in Java and why it is useful.

pg 168

SOLUTION

Object Reflection is a feature in Java that provides a way to get reflective information about Java classes and objects, and perform operations such as:

1. Getting information about the methods and fields present inside the class at runtime.
2. Creating a new instance of a class.
3. Getting and setting the object fields directly by getting field reference, regardless of what the access modifier is.

The code below offers an example of object reflection.

```

1  /* Parameters */
2  Object[] doubleArgs = new Object[] { 4.2, 3.9 };
3
4  /* Get class */
5  Class rectangleDefinition = Class.forName("MyProj.Rectangle");
6

```



```

7  /* Equivalent: Rectangle rectangle = new Rectangle(4.2, 3.9); */
8  Class[] doubleArgsClass = new Class[] {double.class, double.class};
9  Constructor doubleArgsConstructor =
10     rectangleDefinition.getConstructor(doubleArgsClass);
11  Rectangle rectangle = (Rectangle) doubleArgsConstructor.newInstance(doubleArgs);
12
13  /* Equivalent: Double area = rectangle.area(); */
14  Method m = rectangleDefinition.getDeclaredMethod("area");
15  Double area = (Double) m.invoke(rectangle);

```

This code does the equivalent of:

```

1  Rectangle rectangle = new Rectangle(4.2, 3.9);
2  Double area = rectangle.area();

```

Why Is Object Reflection Useful?

Of course, it doesn't seem very useful in the above example, but reflection can be very useful in some cases. Three main reasons are:

1. It can help you observe or manipulate the runtime behavior of applications.
2. It can help you debug or test programs, as you have direct access to methods, constructors, and fields.
3. You can call methods by name when you don't know the method in advance. For example, we may let the user pass in a class name, parameters for the constructor, and a method name. We can then use this information to create an object and call a method. Doing these operations without reflection would require a complex series of if-statements, if it's possible at all.

13.7 Lambda Expressions: There is a class `Country` that has methods `getContinent()` and `getPopulation()`. Write a function `int getPopulation(List<Country> countries, String continent)` that computes the total population of a given continent, given a list of all countries and the name of a continent.

pg 168

SOLUTION

This question really comes in two parts. First, we need to generate a list of the countries in North America. Then, we need to compute their total population.

Without lambda expressions, this is fairly straightforward to do.

```

1  int getPopulation(List<Country> countries, String continent) {
2      int sum = 0;
3      for (Country c : countries) {
4          if (c.getContinent().equals(continent)) {
5              sum += c.getPopulation();
6          }
7      }
8      return sum;
9  }

```

To implement this with lambda expressions, let's break this up into multiple parts.

First, we use `filter` to get a list of the countries in the specified continent.

```

1  Stream<Country> northAmerica = countries.stream().filter(
2      country -> { return country.getContinent().equals(continent);}

```

```
3 );
```

Second, we convert this into a list of populations using `map`.

```
1 Stream<Integer> populations = northAmerica.map(
2   c -> c.getPopulation()
3 );
```

Third and finally, we compute the sum using `reduce`.

```
1 int population = populations.reduce(0, (a, b) -> a + b);
```

This function puts it all together.

```
1 int getPopulation(List<Country> countries, String continent) {
2   /* Filter countries. */
3   Stream<Country> sublist = countries.stream().filter(
4     country -> { return country.getContinent().equals(continent); }
5 );
6
7   /* Convert to list of populations. */
8   Stream<Integer> populations = sublist.map(
9     c -> c.getPopulation()
10  );
11
12  /* Sum list. */
13  int population = populations.reduce(0, (a, b) -> a + b);
14  return population;
15 }
```

Alternatively, because of the nature of this specific problem, we can actually remove the `filter` entirely. The `reduce` operation can have logic that maps the population of countries not in the right continent to zero. The sum will effectively disregard countries not within `continent`.

```
1 int getPopulation(List<Country> countries, String continent) {
2   Stream<Integer> populations = countries.stream().map(
3     c -> c.getContinent().equals(continent) ? c.getPopulation() : 0;
4   return populations.reduce(0, (a, b) -> a + b);
5 }
```

Lambda functions were new to Java 8, so if you don't recognize them, that's probably why. Now is a great time to learn about them, though!

13.8 Lambda Random: Using Lambda expressions, write a function `List<Integer> getRandomSubset(List<Integer> list)` that returns a random subset of arbitrary size. All subsets (including the empty set) should be equally likely to be chosen.

pg 439

SOLUTION

It's tempting to approach this problem by picking a subset size from 0 to N and then generating a random subset of that size.

That creates two issues:

1. We'd have to weight those probabilities. If $N > 1$, there are more subsets of size $N/2$ than there are of subsets of size N (of which there is always only one).
2. It's actually more difficult to generate a subset of a restricted size (e.g., specifically 10) than it is to generate a subset of any size.

Instead, rather than generating a subset based on sizes, let's think about it based on elements. (The fact that we're told to use lambda expressions is also a hint that we should think about some sort of iteration or processing through the elements.)

Imagine we were iterating through {1, 2, 3} to generate a subset. Should 1 be in this subset?

We've got two choices: yes or no. We need to weight the probability of "yes" vs. "no" based on the percent of subsets that contain 1. So, what percent of elements contain 1?

For any specific element, there are as many subsets that contain the element as do not contain it. Consider the following:

{}	{1}
{2}	{1, 2}
{3}	{1, 3}
{2, 3}	{1, 2, 3}

Note how the difference between the subsets on the left and the subsets on the right is the existence of 1. The left and right sides must have the same number of subsets because we can convert from one to the other by just adding an element.

This means that we can generate a random subset by iterating through the list and flipping a coin (i.e., deciding on a 50/50 chance) to pick whether or not each element will be in it.

Without lambda expressions, we can write something like this:

```
1 List<Integer> getRandomSubset(List<Integer> list) {
2     List<Integer> subset = new ArrayList<Integer>();
3     Random random = new Random();
4     for (int item : list) {
5         /* Flip coin. */
6         if (random.nextBoolean()) {
7             subset.add(item);
8         }
9     }
10    return subset;
11 }
```

To implement this approach using lambda expressions, we can do the following:

```
1 List<Integer> getRandomSubset(List<Integer> list) {
2     Random random = new Random();
3     List<Integer> subset = list.stream().filter(
4         k -> { return random.nextBoolean(); /* Flip coin. */
5     }).collect(Collectors.toList());
6     return subset;
7 }
```

Or, we can use a predicate (defined within the class or within the function):

```
1 Random random = new Random();
2 Predicate<Object> flipCoin = o -> {
3     return random.nextBoolean();
4 };
5
6 List<Integer> getRandomSubset(List<Integer> list) {
7     List<Integer> subset = list.stream().filter(flipCoin).
8         collect(Collectors.toList());
9     return subset;
10 }
```

The nice thing about this implementation is that now we can apply the `flipCoin` predicate in other places.

14

Solutions to Databases

Questions 1 through 3 refer to the following database schema:

Apartments	
AptID	int
UnitNumber	varchar(10)
BuildingID	int

Buildings	
BuildingID	int
ComplexID	int
BuildingName	varchar(100)
Address	varchar(500)

Requests	
RequestID	int
Status	varchar(100)
AptID	int
Description	varchar(500)

Complexes	
ComplexID	int
ComplexName	varchar(100)

AptTenants	
TenantID	int
AptID	int

Tenants	
TenantID	int
TenantName	varchar(100)

Note that each apartment can have multiple tenants, and each tenant can have multiple apartments. Each apartment belongs to one building, and each building belongs to one complex.

14.1 Multiple Apartments: Write a SQL query to get a list of tenants who are renting more than one apartment.

pg 172

SOLUTION

To implement this, we can use the **HAVING** and **GROUP BY** clauses and then perform an **INNER JOIN** with **Tenants**.

```
1 SELECT TenantName
2 FROM Tenants
3 INNER JOIN
4     (SELECT TenantID FROM AptTenants GROUP BY TenantID HAVING count(*) > 1) C
5 ON Tenants.TenantID = C.TenantID
```

Whenever you write a **GROUP BY** clause in an interview (or in real life), make sure that anything in the **SELECT** clause is either an aggregate function or contained within the **GROUP BY** clause.

14.2 Open Requests: Write a SQL query to get a list of all buildings and the number of open requests (Requests in which status equals 'Open').

pg 173

SOLUTION

This problem uses a straightforward join of Requests and Apartments to get a list of building IDs and the number of open requests. Once we have this list, we join it again with the Buildings table.

```
1 SELECT BuildingName, ISNULL(Count, 0) as 'Count'
2 FROM Buildings
3 LEFT JOIN
4     (SELECT Apartments.BuildingID, count(*) as 'Count'
5      FROM Requests INNER JOIN Apartments
6      ON Requests.AptID = Apartments.AptID
7      WHERE Requests.Status = 'Open'
8      GROUP BY Apartments.BuildingID) ReqCounts
9 ON ReqCounts.BuildingID = Buildings.BuildingID
```

Queries like this that utilize sub-queries should be thoroughly tested, even when coding by hand. It may be useful to test the inner part of the query first, and then test the outer part.

14.3 Close All Requests: Building #11 is undergoing a major renovation. Implement a query to close all requests from apartments in this building.

pg 173

SOLUTION

UPDATE queries, like SELECT queries, can have WHERE clauses. To implement this query, we get a list of all apartment IDs within building #11 and the list of update requests from those apartments.

```
1 UPDATE Requests
2 SET Status = 'Closed'
3 WHERE AptID IN (SELECT AptID FROM Apartments WHERE BuildingID = 11)
```

14.4 Joins: What are the different types of joins? Please explain how they differ and why certain types are better in certain situations.

pg 173

SOLUTION

JOIN is used to combine the results of two tables. To perform a JOIN, each of the tables must have at least one field that will be used to find matching records from the other table. The join type defines which records will go into the result set.

Let's take for example two tables: one table lists the "regular" beverages, and another lists the calorie-free beverages. Each table has two fields: the beverage name and its product code. The "code" field will be used to perform the record matching.

Regular Beverages:

Name	Code
Budweiser	BUDWEISER
Coca-Cola	COCACOLA

Name	Code
Pepsi	PEPSI

Calorie-Free Beverages:

Name	Code
Diet Coca-Cola	COCACOLA
Fresca	FRESCA
Diet Pepsi	PEPSI
Pepsi Light	PEPSI
Purified Water	Water

If we wanted to join Beverage with Calorie-Free Beverages, we would have many options. These are discussed below.

- **INNER JOIN:** The result set would contain only the data where the criteria match. In our example, we would get three records: one with a COCACOLA code and two with PEPSI codes.
- **OUTER JOIN:** An OUTER JOIN will always contain the results of INNER JOIN, but it may also contain some records that have no matching record in the other table. OUTER JOINS are divided into the following subtypes:
 - » **LEFT OUTER JOIN, or simply LEFT JOIN:** The result will contain all records from the left table. If no matching records were found in the right table, then its fields will contain the NULL values. In our example, we would get four records. In addition to INNER JOIN results, BUDWEISER would be listed, because it was in the left table.
 - » **RIGHT OUTER JOIN, or simply RIGHT JOIN:** This type of join is the opposite of LEFT JOIN. It will contain every record from the right table; the missing fields from the left table will be NULL. Note that if we have two tables, A and B, then we can say that the statement A LEFT JOIN B is equivalent to the statement B RIGHT JOIN A. In our example above, we will get five records. In addition to INNER JOIN results, FRESCA and WATER records will be listed.
 - » **FULL OUTER JOIN:** This type of join combines the results of the LEFT and RIGHT JOINS. All records from both tables will be included in the result set, regardless of whether or not a matching record exists in the other table. If no matching record was found, then the corresponding result fields will have a NULL value. In our example, we will get six records.

14.5 Denormalization: What is denormalization? Explain the pros and cons.

pg 173

SOLUTION

Denormalization is a database optimization technique in which we add redundant data to one or more tables. This can help us avoid costly joins in a relational database.

By contrast, in a traditional normalized database, we store data in separate logical tables and attempt to minimize redundant data. We may strive to have only one copy of each piece of data in the database.

For example, in a normalized database, we might have a Courses table and a Teachers table. Each entry in Courses would store the teacherID for a Course but not the teacherName. When we need to retrieve a list of all Courses with the Teacher name, we would do a join between these two tables.

In some ways, this is great; if a teacher changes his or her name, we only have to update the name in one place.

The drawback, however, is that if the tables are large, we may spend an unnecessarily long time doing joins on tables.

Denormalization, then, strikes a different compromise. Under denormalization, we decide that we're okay with some redundancy and some extra effort to update the database in order to get the efficiency advantages of fewer joins.

Cons of Denormalization	Pros of Denormalization
Updates and inserts are more expensive.	Retrieving data is faster since we do fewer joins.
Denormalization can make update and insert code harder to write.	Queries to retrieve can be simpler (and therefore less likely to have bugs), since we need to look at fewer tables.
Data may be inconsistent. Which is the "correct" value for a piece of data?	
Data redundancy necessitates more storage.	

In a system that demands scalability, like that of any major tech companies, we almost always use elements of both normalized and denormalized databases.

14.6 Entity-Relationship Diagram: Draw an entity-relationship diagram for a database with companies, people, and professionals (people who work for companies).

pg 173

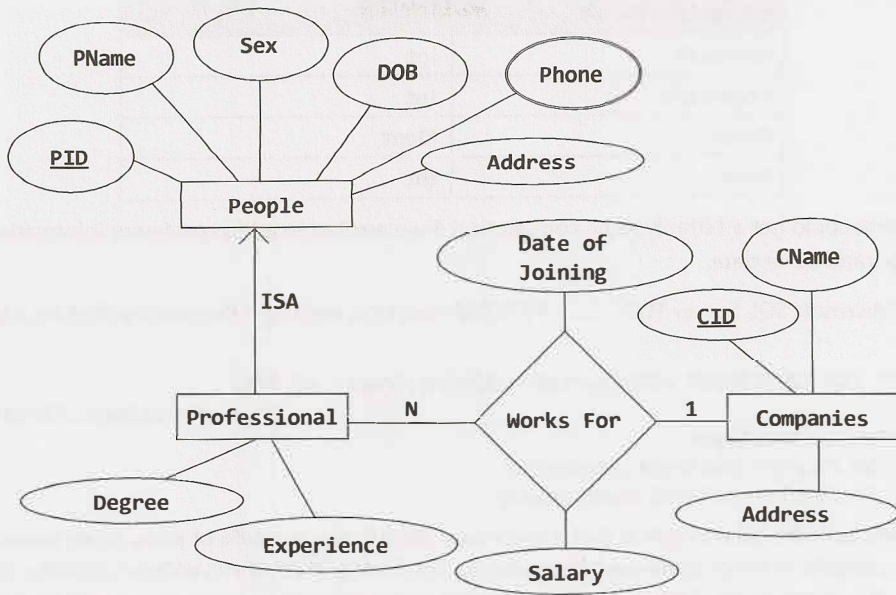
SOLUTION

People who work for Companies are Professionals. So, there is an ISA ("is a") relationship between People and Professionals (or we could say that a Professional is derived from People).

Each Professional has additional information such as degree and work experiences in addition to the properties derived from People.

A Professional works for one company at a time (probably—you might want to validate this assumption), but Companies can hire many Professionals. So, there is a many-to-one relationship between Professionals and Companies. This "Works For" relationship can store attributes such as an employee's start date and salary. These attributes are defined only when we relate a Professional with a Company.

A Person can have multiple phone numbers, which is why Phone is a multi-valued attribute.



14.7 Design Grade Database: Imagine a simple database storing information for students' grades. Design what this database might look like and provide a SQL query to return a list of the honor roll students (top 10%), sorted by their grade point average.

pg 173

SOLUTION

In a simplistic database, we'll have at least three objects: Students, Courses, and CourseEnrollment. Students will have at least a student name and ID and will likely have other personal information. Courses will contain the course name and ID and will likely contain the course description, professor, and other information. CourseEnrollment will pair Students and Courses and will also contain a field for CourseGrade.

Students	
StudentID	int
StudentName	varchar(100)
Address	varchar(500)

Courses	
CourseID	int
CourseName	varchar(100)
ProfessorID	int

CourseEnrollment	
CourseID	int
StudentID	int
Grade	float
Term	int

This database could get arbitrarily more complicated if we wanted to add in professor information, billing information, and other data.

Using the Microsoft SQL Server TOP ... PERCENT function, we might (incorrectly) first try a query like this:

```
1 SELECT TOP 10 PERCENT AVG(CourseEnrollment.Grade) AS GPA,
2                               CourseEnrollment.StudentID
3 FROM CourseEnrollment
4 GROUP BY CourseEnrollment.StudentID
5 ORDER BY AVG(CourseEnrollment.Grade)
```

The problem with the above code is that it will return literally the top 10% of rows, when sorted by GPA. Imagine a scenario in which there are 100 students, and the top 15 students all have 4.0 GPAs. The above function will only return 10 of those students, which is not really what we want. In case of a tie, we want to include the students who tied for the top 10% -- even if this means that our honor roll includes more than 10% of the class.

To correct this issue, we can build something similar to this query, but instead first get the GPA cut off.

```
1 DECLARE @GPACutOff float;
2 SET @GPACutOff = (SELECT min(GPA) as 'GPAMin' FROM (
3     SELECT TOP 10 PERCENT AVG(CourseEnrollment.Grade) AS GPA
4     FROM CourseEnrollment
5     GROUP BY CourseEnrollment.StudentID
6     ORDER BY GPA desc) Grades);
```

Then, once we have @GPACutOff defined, selecting the students with at least this GPA is reasonably straightforward.

```
1 SELECT StudentName, GPA
2 FROM (SELECT AVG(CourseEnrollment.Grade) AS GPA, CourseEnrollment.StudentID
3     FROM CourseEnrollment
4     GROUP BY CourseEnrollment.StudentID
5     HAVING AVG(CourseEnrollment.Grade) >= @GPACutOff) Honors
6 INNER JOIN Students ON Honors.StudentID = Student.StudentID
```

Be very careful about what implicit assumptions you make. If you look at the above database description, what potentially incorrect assumption do you see? One is that each course can only be taught by one professor. At some schools, courses may be taught by multiple professors.

However, you *will* need to make some assumptions, or you'd drive yourself crazy. Which assumptions you make is less important than just recognizing *that* you made assumptions. Incorrect assumptions, both in the real world and in an interview, can be dealt with *as long as they are acknowledged*.

Remember, additionally, that there's a trade-off between flexibility and complexity. Creating a system in which a course can have multiple professors does increase the database's flexibility, but it also increases its complexity. If we tried to make our database flexible to every possible situation, we'd wind up with something hopelessly complex.

Make your design reasonably flexible, and state any other assumptions or constraints. This goes for not just database design, but object-oriented design and programming in general.

15

Solutions to Threads and Locks

15.1 Thread vs. Process: What's the difference between a thread and a process?

pg 179

SOLUTION

Processes and threads are related to each other but are fundamentally different.

A process can be thought of as an instance of a program in execution. A process is an independent entity to which system resources (e.g., CPU time and memory) are allocated. Each process is executed in a separate address space, and one process cannot access the variables and data structures of another process. If a process wishes to access another process' resources, inter-process communications have to be used. These include pipes, files, sockets, and other forms.

A thread exists within a process and shares the process' resources (including its heap space). Multiple threads within the same process will share the same heap space. This is very different from processes, which cannot directly access the memory of another process. Each thread still has its own registers and its own stack, but other threads can read and write the heap memory.

A thread is a particular execution path of a process. When one thread modifies a process resource, the change is immediately visible to sibling threads.

15.2 Context Switch: How would you measure the time spent in a context switch?

pg 179

SOLUTION

This is a tricky question, but let's start with a possible solution.

A context switch is the time spent switching between two processes (i.e., bringing a waiting process into execution and sending an executing process into waiting/terminated state). This happens in multitasking. The operating system must bring the state information of waiting processes into memory and save the state information of the currently running process.

In order to solve this problem, we would like to record the timestamps of the last and first instruction of the swapping processes. The context switch time is the difference in the timestamps between the two processes.

Let's take an easy example: Assume there are only two processes, P_1 and P_2 .

P_1 is executing and P_2 is waiting for execution. At some point, the operating system must swap P_1 and P_2 —let's assume it happens at the N th instruction of P_1 . If $t_{x,k}$ indicates the timestamp in microseconds of the k th instruction of process x , then the context switch would take $t_{2,1} - t_{1,n}$ microseconds.

The tricky part is this: how do we know when this swapping occurs? We cannot, of course, record the timestamp of every instruction in the process.

Another issue is that swapping is governed by the scheduling algorithm of the operating system and there may be many kernel level threads which are also doing context switches. Other processes could be contending for the CPU or the kernel handling interrupts. The user does not have any control over these extraneous context switches. For instance, if at time $t_{1,n}$ the kernel decides to handle an interrupt, then the context switch time would be overstated.

In order to overcome these obstacles, we must first construct an environment such that after P_1 executes, the task scheduler immediately selects P_2 to run. This may be accomplished by constructing a data channel, such as a pipe, between P_1 and P_2 and having the two processes play a game of ping-pong with a data token.

That is, let's allow P_1 to be the initial sender and P_2 to be the receiver. Initially, P_2 is blocked (sleeping) as it awaits the data token. When P_1 executes, it delivers the token over the data channel to P_2 and immediately attempts to read a response token. However, since P_2 has not yet had a chance to run, no such token is available for P_1 and the process is blocked. This relinquishes the CPU.

A context switch results and the task scheduler must select another process to run. Since P_2 is now in a ready-to-run state, it is a desirable candidate to be selected by the task scheduler for execution. When P_2 runs, the roles of P_1 and P_2 are swapped. P_2 is now acting as the sender and P_1 as the blocked receiver. The game ends when P_2 returns the token to P_1 .

To summarize, an iteration of the game is played with the following steps:

1. P_2 blocks awaiting data from P_1 .
2. P_1 marks the start time.
3. P_1 sends token to P_2 .
4. P_1 attempts to read a response token from P_2 . This induces a context switch.
5. P_2 is scheduled and receives the token.
6. P_2 sends a response token to P_1 .
7. P_2 attempts read a response token from P_1 . This induces a context switch.
8. P_1 is scheduled and receives the token.
9. P_1 marks the end time.

The key is that the delivery of a data token induces a context switch. Let T_d and T_r be the time it takes to deliver and receive a data token, respectively, and let T_c be the amount of time spent in a context switch. At step 2, P_1 records the timestamp of the delivery of the token, and at step 9, it records the timestamp of the response. The amount of time elapsed, T , between these events may be expressed by:

$$T = 2 * (T_d + T_c + T_r)$$

This formula arises because of the following events: P_1 sends a token (3), the CPU context switches (4), P_2 receives it (5). P_2 then sends the response token (6), the CPU context switches (7), and finally P_1 receives it (8).

P_1 will be able to easily compute T , since this is just the time between events 3 and 8. So, to solve for T_c , we must first determine the value of $T_d + T_r$.

How can we do this? We can do this by measuring the length of time it takes P_1 to send and receive a token to itself. This will not induce a context switch since P_1 is running on the CPU at the time it sent the token and will not block to receive it.

The game is played a number of iterations to average out any variability in the elapsed time between steps 2 and 9 that may result from unexpected kernel interrupts and additional kernel threads contending for the CPU. We select the smallest observed context switch time as our final answer.

However, all we can ultimately say that this is an approximation which depends on the underlying system. For example, we make the assumption that P_2 is selected to run once a data token becomes available. However, this is dependent on the implementation of the task scheduler and we cannot make any guarantees.

That's okay; it's important in an interview to recognize when your solution might not be perfect.

15.3 Dining Philosophers: In the famous dining philosophers problem, a bunch of philosophers are sitting around a circular table with one chopstick between each of them. A philosopher needs both chopsticks to eat, and always picks up the left chopstick before the right one. A deadlock could potentially occur if all the philosophers reached for the left chopstick at the same time. Using threads and locks, implement a simulation of the dining philosophers problem that prevents deadlocks.

pg 180

SOLUTION

First, let's implement a simple simulation of the dining philosophers problem in which we don't concern ourselves with deadlocks. We can implement this solution by having `Philosopher` extend `Thread`, and `Chopstick` call `lock.lock()` when it is picked up and `lock.unlock()` when it is put down.

```

1  class Chopstick {
2      private Lock lock;
3
4      public Chopstick() {
5          lock = new ReentrantLock();
6      }
7
8      public void pickUp() {
9          lock.lock();
10     }
11
12     public void putDown() {
13         lock.unlock();
14     }
15 }
16
17 class Philosopher extends Thread {
18     private int bites = 10;
19     private Chopstick left, right;
20
21     public Philosopher(Chopstick left, Chopstick right) {
22         this.left = left;
23         this.right = right;
24     }

```

```
25
26     public void eat() {
27         pickUp();
28         chew();
29         putDown();
30     }
31
32     public void pickUp() {
33         left.pickUp();
34         right.pickUp();
35     }
36
37     public void chew() { }
38
39     public void putDown() {
40         right.putDown();
41         left.putDown();
42     }
43
44     public void run() {
45         for (int i = 0; i < bites; i++) {
46             eat();
47         }
48     }
49 }
```

Running the above code may lead to a deadlock if all the philosophers have a left chopstick and are waiting for the right one.

Solution #1: All or Nothing

To prevent deadlocks, we can implement a strategy where a philosopher will put down his left chopstick if he is unable to obtain the right one.

```
1  public class Chopstick {
2      /* same as before */
3
4      public boolean pickUp() {
5          return lock.tryLock();
6      }
7  }
8
9  public class Philosopher extends Thread {
10     /* same as before */
11
12     public void eat() {
13         if (pickUp()) {
14             chew();
15             putDown();
16         }
17     }
18
19     public boolean pickUp() {
20         /* attempt to pick up */
21         if (!left.pickUp()) {
22             return false;
23         }
24         if (!right.pickUp()) {
```

```

25         left.putDown();
26         return false;
27     }
28     return true;
29 }
30 }

```

In the above code, we need to be sure to release the left chopstick if we can't pick up the right one—and to not call `putDown()` on the chopsticks if we never had them in the first place.

One issue with this is that if all the philosophers were perfectly synchronized, they could simultaneously pick up their left chopstick, be unable to pick up the right one, and then put back down the left one—only to have the process repeated again.

Solution #2: Prioritized Chopsticks

Alternatively, we can label the chopsticks with a number from 0 to $N - 1$. Each philosopher attempts to pick up the lower numbered chopstick first. This essentially means that each philosopher goes for the left chopstick before right one (assuming that's the way you labeled it), except for the last philosopher who does this in reverse. This will break the cycle.

```

1  public class Philosopher extends Thread {
2      private int bites = 10;
3      private Chopstick lower, higher;
4      private int index;
5      public Philosopher(int i, Chopstick left, Chopstick right) {
6          index = i;
7          if (left.getNumber() < right.getNumber()) {
8              this.lower = left;
9              this.higher = right;
10         } else {
11             this.lower = right;
12             this.higher = left;
13         }
14     }
15
16     public void eat() {
17         pickUp();
18         chew();
19         putDown();
20     }
21
22     public void pickUp() {
23         lower.pickUp();
24         higher.pickUp();
25     }
26
27     public void chew() { ... }
28
29     public void putDown() {
30         higher.putDown();
31         lower.putDown();
32     }
33
34     public void run() {
35         for (int i = 0; i < bites; i++) {
36             eat();

```



```
37     }
38 }
39 }
40
41 public class Chopstick {
42     private Lock lock;
43     private int number;
44
45     public Chopstick(int n) {
46         lock = new ReentrantLock();
47         this.number = n;
48     }
49
50     public void pickUp() {
51         lock.lock();
52     }
53
54     public void putDown() {
55         lock.unlock();
56     }
57
58     public int getNumber() {
59         return number;
60     }
61 }
```

With this solution, a philosopher can never hold the larger chopstick without holding the smaller one. This prevents the ability to have a cycle, since a cycle means that a higher chopstick would “point” to a lower one.

15.4 Deadlock-Free Class: Design a class which provides a lock only if there are no possible deadlocks.

pg 180

SOLUTION

There are several common ways to prevent deadlocks. One of the popular ways is to require a process to declare upfront what locks it will need. We can then verify if a deadlock would be created by issuing these locks, and we can fail if so.

With these constraints in mind, let’s investigate how we can detect deadlocks. Suppose this was the order of locks requested:

```
A = {1, 2, 3, 4}
B = {1, 3, 5}
C = {7, 5, 9, 2}
```

This may create a deadlock because we could have the following scenario:

```
A locks 2, waits on 3
B locks 3, waits on 5
C locks 5, waits on 2
```

We can think about this as a graph, where 2 is connected to 3, 3 is connected to 5, and 5 is connected to 2. A deadlock is represented by a cycle. An edge (w, v) exists in the graph if a process declares that it will request lock v immediately after lock w . For the earlier example, the following edges would exist in the graph: (1, 2), (2, 3), (3, 4), (1, 3), (3, 5), (7, 5), (5, 9), (9, 2). The “owner” of the edge does not matter.

This class will need a declare method, which threads and processes will use to declare what order they will request resources in. This declare method will iterate through the declare order, adding each contiguous pair of elements (v, w) to the graph. Afterwards, it will check to see if any cycles have been created. If any cycles have been created, it will backtrack, removing these edges from the graph, and then exit.

We have one final component to discuss: how do we detect a cycle? We can detect a cycle by doing a depth-first search through each connected component (i.e., each connected part of the graph). Complex algorithms exist to find all the connected components of a graph, but our work in this problem does not require this degree of complexity.

We know that if a cycle was created, one of our new edges must be to blame. Thus, as long as our depth-first search touches all of these edges at some point, then we know that we have fully searched for a cycle.

The pseudocode for this special case cycle detection looks like this:

```

1  boolean checkForCycle(locks[] locks) {
2      touchedNodes = hash table(lock -> boolean)
3      initialize touchedNodes to false for each lock in locks
4      for each (lock x in process.locks) {
5          if (touchedNodes[x] == false) {
6              if (hasCycle(x, touchedNodes)) {
7                  return true;
8              }
9          }
10     }
11     return false;
12 }
13
14 boolean hasCycle(node x, touchedNodes) {
15     touchedNodes[x] = true;
16     if (x.state == VISITING) {
17         return true;
18     } else if (x.state == FRESH) {
19         ... (see full code below)
20     }
21 }
```

In the above code, note that we may do several depth-first searches, but touchedNodes is only initialized once. We iterate until all the values in touchedNodes are false.

The code below provides further details. For simplicity, we assume that all locks and processes (owners) are ordered sequentially.

```

1  class LockFactory {
2      private static LockFactory instance;
3
4      private int numberOfLocks = 5; /* default */
5      private LockNode[] locks;
6
7      /* Maps from a process or owner to the order that the owner claimed it would
8       * call the locks in */
9      private HashMap<Integer, LinkedList<LockNode>> lockOrder;
10
11     private LockFactory(int count) { ... }
12     public static LockFactory getInstance() { return instance; }
13
14     public static synchronized LockFactory initialize(int count) {
15         if (instance == null) instance = new LockFactory(count);
16     }
17 }
```

```

16     return instance;
17 }
18
19 public boolean hasCycle(HashMap<Integer, Boolean> touchedNodes,
20                        int[] resourcesInOrder) {
21     /*check for a cycle */
22     for (int resource : resourcesInOrder) {
23         if (touchedNodes.get(resource) == false) {
24             LockNode n = locks[resource];
25             if (n.hasCycle(touchedNodes)) {
26                 return true;
27             }
28         }
29     }
30     return false;
31 }
32
33 /*To prevent deadlocks, force the processes to declare upfront what order they
34 * will need the locks in. Verify that this order does not create a deadlock (a
35 * cycle in a directed graph) */
36 public boolean declare(int ownerId, int[] resourcesInOrder) {
37     HashMap<Integer, Boolean> touchedNodes = new HashMap<Integer, Boolean>();
38
39     /*add nodes to graph */
40     int index = 1;
41     touchedNodes.put(resourcesInOrder[0], false);
42     for (index = 1; index < resourcesInOrder.length; index++) {
43         LockNode prev = locks[resourcesInOrder[index - 1]];
44         LockNode curr = locks[resourcesInOrder[index]];
45         prev.joinTo(curr);
46         touchedNodes.put(resourcesInOrder[index], false);
47     }
48
49     /*if we created a cycle, destroy this resource list and return false */
50     if (hasCycle(touchedNodes, resourcesInOrder)) {
51         for (int j = 1; j < resourcesInOrder.length; j++) {
52             LockNode p = locks[resourcesInOrder[j - 1]];
53             LockNode c = locks[resourcesInOrder[j]];
54             p.remove(c);
55         }
56         return false;
57     }
58
59     /*No cycles detected. Save the order that was declared, so that we can
60     * verify that the process is really calling the locks in the order it said
61     * it would. */
62     LinkedList<LockNode> list = new LinkedList<LockNode>();
63     for (int i = 0; i < resourcesInOrder.length; i++) {
64         LockNode resource = locks[resourcesInOrder[i]];
65         list.add(resource);
66     }
67     lockOrder.put(ownerId, list);
68
69     return true;
70 }
71

```

```

72  /* Get the lock, verifying first that the process is really calling the locks in
73  * the order it said it would. */
74  public Lock getLock(int ownerId, int resourceID) {
75      LinkedList<LockNode> list = lockOrder.get(ownerId);
76      if (list == null) return null;
77
78      LockNode head = list.getFirst();
79      if (head.getId() == resourceID) {
80          list.removeFirst();
81          return head.getLock();
82      }
83      return null;
84  }
85  }
86
87  public class LockNode {
88      public enum VisitState { FRESH, VISITING, VISITED };
89
90      private ArrayList<LockNode> children;
91      private int lockId;
92      private Lock lock;
93      private int maxLocks;
94
95      public LockNode(int id, int max) { ... }
96
97      /* Join "this" to "node", checking that it doesn't create a cycle */
98      public void joinTo(LockNode node) { children.add(node); }
99      public void remove(LockNode node) { children.remove(node); }
100
101      /* Check for a cycle by doing a depth-first-search. */
102      public boolean hasCycle(HashMap<Integer, Boolean> touchedNodes) {
103          VisitState[] visited = new VisitState[maxLocks];
104          for (int i = 0; i < maxLocks; i++) {
105              visited[i] = VisitState.FRESH;
106          }
107          return hasCycle(visited, touchedNodes);
108      }
109
110      private boolean hasCycle(VisitState[] visited,
111                              HashMap<Integer, Boolean> touchedNodes) {
112          if (touchedNodes.containsKey(lockId)) {
113              touchedNodes.put(lockId, true);
114          }
115
116          if (visited[lockId] == VisitState.VISITING) {
117              /* We looped back to this node while still visiting it, so we know there's
118               * a cycle. */
119              return true;
120          } else if (visited[lockId] == VisitState.FRESH) {
121              visited[lockId] = VisitState.VISITING;
122              for (LockNode n : children) {
123                  if (n.hasCycle(visited, touchedNodes)) {
124                      return true;
125                  }
126              }
127              visited[lockId] = VisitState.VISITED;

```

```
128     }
129     return false;
130 }
131
132 public Lock getLock() {
133     if (lock == null) lock = new ReentrantLock();
134     return lock;
135 }
136
137 public int getId() { return lockId; }
138 }
```

As always, when you see code this complicated and lengthy, you wouldn't be expected to write all of it. More likely, you would be asked to sketch out pseudocode and possibly implement one of these methods.

15.5 Call In Order: Suppose we have the following code:

```
public class Foo {
    public Foo() { ... }
    public void first() { ... }
    public void second() { ... }
    public void third() { ... }
}
```

The same instance of `Foo` will be passed to three different threads. `ThreadA` will call `first`, `threadB` will call `second`, and `threadC` will call `third`. Design a mechanism to ensure that `first` is called before `second` and `second` is called before `third`.

pg 180

SOLUTION

The general logic is to check if `first()` has completed before executing `second()`, and if `second()` has completed before calling `third()`. Because we need to be very careful about thread safety, simple boolean flags won't do the job.

What about using a lock to do something like the below code?

```
1 public class FooBad {
2     public int pauseTime = 1000;
3     public ReentrantLock lock1, lock2;
4
5     public FooBad() {
6         try {
7             lock1 = new ReentrantLock();
8             lock2 = new ReentrantLock();
9
10            lock1.lock();
11            lock2.lock();
12        } catch (...) { ... }
13    }
14
15    public void first() {
16        try {
17            ...
18            lock1.unlock(); // mark finished with first()
19        } catch (...) { ... }
20    }
```



```

21
22     public void second() {
23         try {
24             lock1.lock(); // wait until finished with first()
25             lock1.unlock();
26             ...
27
28             lock2.unlock(); // mark finished with second()
29         } catch (...) { ... }
30     }
31
32     public void third() {
33         try {
34             lock2.lock(); // wait until finished with third()
35             lock2.unlock();
36             ...
37         } catch (...) { ... }
38     }
39 }

```

This code won't actually quite work due to the concept of *lock ownership*. One thread is actually performing the lock (in the `FooBad` constructor), but different threads attempt to unlock the locks. This is not allowed, and your code will raise an exception. A lock in Java is owned by the same thread which locked it.

Instead, we can replicate this behavior with semaphores. The logic is identical.

```

1  public class Foo {
2      public Semaphore sem1, sem2;
3
4      public Foo() {
5          try {
6              sem1 = new Semaphore(1);
7              sem2 = new Semaphore(1);
8
9              sem1.acquire();
10             sem2.acquire();
11         } catch (...) { ... }
12     }
13
14     public void first() {
15         try {
16             ...
17             sem1.release();
18         } catch (...) { ... }
19     }
20
21     public void second() {
22         try {
23             sem1.acquire();
24             sem1.release();
25             ...
26             sem2.release();
27         } catch (...) { ... }
28     }
29
30     public void third() {
31         try {
32             sem2.acquire();

```

```
33         sem2.release();
34         ...
35     } catch (...) { ... }
36 }
37 }
```

15.6 Synchronized Methods: You are given a class with synchronized method A and a normal method B. If you have two threads in one instance of a program, can they both execute A at the same time? Can they execute A and B at the same time?

pg 180

SOLUTION

By applying the word *synchronized* to a method, we ensure that two threads cannot execute synchronized methods *on the same object instance* at the same time.

So, the answer to the first part really depends. If the two threads have the same instance of the object, then no, they cannot simultaneously execute method A. However, if they have different instances of the object, then they can.

Conceptually, you can see this by considering locks. A synchronized method applies a “lock” on *all* synchronized methods in that instance of the object. This blocks other threads from executing synchronized methods within that instance.

In the second part, we’re asked if thread1 can execute synchronized method A while thread2 is executing non-synchronized method B. Since B is not synchronized, there is nothing to block thread1 from executing A while thread2 is executing B. This is true regardless of whether thread1 and thread2 have the same instance of the object.

Ultimately, the key concept to remember is that only one synchronized method can be in execution per instance of that object. Other threads can execute non-synchronized methods on that instance, or they can execute any method on a different instance of the object.

15.7 FizzBuzz: In the classic problem FizzBuzz, you are told to print the numbers from 1 to n. However, when the number is divisible by 3, print “Fizz”. When it is divisible by 5, print “Buzz”. When it is divisible by 3 and 5, print “FizzBuzz”. In this problem, you are asked to do this in a multithreaded way. Implement a multithreaded version of FizzBuzz with four threads. One thread checks for divisibility of 3 and prints “Fizz”. Another thread is responsible for divisibility of 5 and prints “Buzz”. A third thread is responsible for divisibility of 3 and 5 and prints “FizzBuzz”. A fourth thread does the numbers.

pg 180

SOLUTION

Let’s start off with implementing a single threaded version of FizzBuzz.

Single Threaded

Although this problem (in the single threaded version) shouldn’t be hard, a lot of candidates overcomplicate it. They look for something “beautiful” that reuses the fact that the divisible by 3 and 5 case (“FizzBuzz”) seems to resemble the individual cases (“Fizz” and “Buzz”).

In actuality, the best way to do it, considering readability and efficiency, is just the straightforward way.

```
1 void fizzbuzz(int n) {
```

```

2  for (int i = 1; i <= n; i++) {
3      if (i % 3 == 0 && i % 5 == 0) {
4          System.out.println("FizzBuzz");
5      } else if (i % 3 == 0) {
6          System.out.println("Fizz");
7      } else if (i % 5 == 0) {
8          System.out.println("Buzz");
9      } else {
10         System.out.println(i);
11     }
12 }
13 }

```

The primary thing to be careful of here is the order of the statements. If you put the check for divisibility by 3 before the check for divisibility by 3 and 5, it won't print the right thing.

Multithreaded

To do this multithreaded, we want a structure that looks something like this:

FizzBuzz Thread	Fizz Thread
if i div by 3 && 5 print FizzBuzz increment i repeat until i > n	if i div by only 3 print Fizz increment i repeat until i > n
Buzz Thread	Number Thread
if i div by only 5 print Buzz increment i repeat until i > n	if i not div by 3 or 5 print i increment i repeat until i > n

The code for this will look something like:

```

1  while (true) {
2      if (current > max) {
3          return;
4      }
5      if (/* divisibility test */) {
6          System.out.println(/* print something */);
7          current++;
8      }
9  }

```

We'll need to add some synchronization in the loop. Otherwise, the value of `current` could change between lines 2 - 4 and lines 5 - 8, and we can inadvertently exceed the intended bounds of the loop. Additionally, incrementing is not thread-safe.

To actually implement this concept, there are many possibilities. One possibility is to have four entirely separate thread classes that share a reference to the `current` variable (which can be wrapped in an object).

The loop for each thread is substantially similar. They just have different target values for the divisibility checks, and different print values.

	FizzBuzz	Fizz	Buzz	Number
current % 3 == 0	true	true	false	false
current % 5 == 0	true	false	true	false
to print	FizzBuzz	Fizz	Buzz	current

For the most part, this can be handled by taking in “target” parameters and the value to print. The output for the Number thread needs to be overwritten, though, as it’s not a simple, fixed string.

We can implement a FizzBuzzThread class which handles most of this. A NumberThread class can extend FizzBuzzThread and override the print method.

```

1  Thread[] threads = {new FizzBuzzThread(true, true, n, "FizzBuzz"),
2                      new FizzBuzzThread(true, false, n, "Fizz"),
3                      new FizzBuzzThread(false, true, n, "Buzz"),
4                      new NumberThread(false, false, n)};
5  for (Thread thread : threads) {
6      thread.start();
7  }
8
9  public class FizzBuzzThread extends Thread {
10     private static Object lock = new Object();
11     protected static int current = 1;
12     private int max;
13     private boolean div3, div5;
14     private String toPrint;
15
16     public FizzBuzzThread(boolean div3, boolean div5, int max, String toPrint) {
17         this.div3 = div3;
18         this.div5 = div5;
19         this.max = max;
20         this.toPrint = toPrint;
21     }
22
23     public void print() {
24         System.out.println(toPrint);
25     }
26
27     public void run() {
28         while (true) {
29             synchronized (lock) {
30                 if (current > max) {
31                     return;
32                 }
33
34                 if ((current % 3 == 0) == div3 &&
35                     (current % 5 == 0) == div5) {
36                     print();
37                     current++;
38                 }
39             }
40         }
41     }
42 }
43
44 public class NumberThread extends FizzBuzzThread {

```

```

45     public NumberThread(boolean div3, boolean div5, int max) {
46         super(div3, div5, max, null);
47     }
48
49     public void print() {
50         System.out.println(current);
51     }
52 }

```

Observe that we need to put the comparison of `current` and `max` before the `if` statement, to ensure the value will only get printed when `current` is less than or equal to `max`.

Alternatively, if we're working in a language which supports this (Java 8 and many other languages do), we can pass in a `validate` method and a `print` method as parameters.

```

1  int n = 100;
2  Thread[] threads = {
3      new FBThread(i -> i % 3 == 0 && i % 5 == 0, i -> "FizzBuzz", n),
4      new FBThread(i -> i % 3 == 0 && i % 5 != 0, i -> "Fizz", n),
5      new FBThread(i -> i % 3 != 0 && i % 5 == 0, i -> "Buzz", n),
6      new FBThread(i -> i % 3 != 0 && i % 5 != 0, i -> Integer.toString(i), n)};
7  for (Thread thread : threads) {
8      thread.start();
9  }
10
11 public class FBThread extends Thread {
12     private static Object lock = new Object();
13     protected static int current = 1;
14     private int max;
15     private Predicate<Integer> validate;
16     private Function<Integer, String> printer;
17     int x = 1;
18
19     public FBThread(Predicate<Integer> validate,
20                     Function<Integer, String> printer, int max) {
21         this.validate = validate;
22         this.printer = printer;
23         this.max = max;
24     }
25
26     public void run() {
27         while (true) {
28             synchronized (lock) {
29                 if (current > max) {
30                     return;
31                 }
32                 if (validate.test(current)) {
33                     System.out.println(printer.apply(current));
34                     current++;
35                 }
36             }
37         }
38     }
39 }

```

There are of course many other ways of implementing this as well.

16

Solutions to Moderate

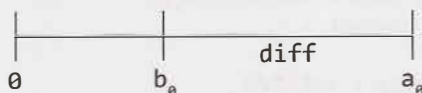
16.1 Number Swapper: Write a function to swap a number in place (that is, without temporary variables).

pg 181

SOLUTION

This is a classic interview problem, and it's a reasonably straightforward one. We'll walk through this using a_0 to indicate the original value of a and b_0 to indicate the original value of b . We'll also use $diff$ to indicate the value of $a_0 - b_0$.

Let's picture these on a number line for the case where $a > b$.



First, we briefly set a to $diff$, which is the right side of the above number line. Then, when we add b and $diff$ (and store that value in b), we get a_0 . We now have $b = a_0$ and $a = diff$. All that's left to do is to set a equal to $a_0 - diff$, which is just $b - a$.

The code below implements this.

```
1 // Example for a = 9, b = 4
2 a = a - b; // a = 9 - 4 = 5
3 b = a + b; // b = 5 + 4 = 9
4 a = b - a; // a = 9 - 5 = 4
```

We can implement a similar solution with bit manipulation. The benefit of this solution is that it works for more data types than just integers.

```
1 // Example for a = 101 (in binary) and b = 110
2 a = a^b; // a = 101^110 = 011
3 b = a^b; // b = 011^110 = 101
4 a = a^b; // a = 011^101 = 110
```

This code works by using XORs. The easiest way to see how this works is by focusing on a specific bit. If we can correctly swap two bits, then we know the entire operation works correctly.

Let's take two bits, x and y , and walk through this line by line.

1. $x = x \oplus y$

This line essentially checks if x and y have different values. It will result in 1 if and only if $x \neq y$.

2. $y = x \oplus y$

Or: $y = \{0 \text{ if originally same, } 1 \text{ if different}\} \wedge \{\text{original } y\}$

Observe that XORing a bit with 1 always flips the bit, whereas XORing with 0 will never change it.

Therefore, if we do $y = 1 \wedge \{\text{original } y\}$ when $x \neq y$, then y will be flipped and therefore have x 's original value.

Otherwise, if $x == y$, then we do $y = 0 \wedge \{\text{original } y\}$ and the value of y does not change.

Either way, y will be equal to the original value of x .

3. $x = x \wedge y$

Or: $x = \{0 \text{ if originally same, } 1 \text{ if different}\} \wedge \{\text{original } x\}$

At this point, y is equal to the original value of x . This line is essentially equivalent to the line above it, but for different variables.

If we do $x = 1 \wedge \{\text{original } x\}$ when the values are different, x will be flipped.

If we do $x = 0 \wedge \{\text{original } x\}$ when the values are the same, x will not be changed.

This operation happens for each bit. Since it correctly swaps each bit, it will correctly swap the entire number.

16.2 Word Frequencies: Design a method to find the frequency of occurrences of any given word in a book. What if we were running this algorithm multiple times?

pg 181

SOLUTION

Let's start with the simple case.

Solution: Single Query

In this case, we simply go through the book, word by word, and count the number of times that a word appears. This will take $O(n)$ time. We know we can't do better than that since we must look at every word in the book.

```
1 int getFrequency(String[] book, String word) {
2     word = word.trim().toLowerCase();
3     int count = 0;
4     for (String w : book) {
5         if (w.trim().toLowerCase().equals(word)) {
6             count++;
7         }
8     }
9     return count;
10 }
```

We have also converted the string to lowercase and trimmed it. You can discuss with your interviewer if this is necessary (or even desired).

Solution: Repetitive Queries

If we're doing the operation repeatedly, then we can probably afford to take some time and extra memory to do pre-processing on the book. We can create a hash table which maps from a word to its frequency. The frequency of any word can be easily looked up in $O(1)$ time. The code for this is below.

```
1 HashMap<String, Integer> setupDictionary(String[] book) {
2     HashMap<String, Integer> table =
```

```

3      new HashMap<String, Integer>());
4      for (String word : book) {
5          word = word.toLowerCase();
6          if (word.trim() != "") {
7              if (!table.containsKey(word)) {
8                  table.put(word, 0);
9              }
10             table.put(word, table.get(word) + 1);
11         }
12     }
13     return table;
14 }
15
16 int getFrequency(HashMap<String, Integer> table, String word) {
17     if (table == null || word == null) return -1;
18     word = word.toLowerCase();
19     if (table.containsKey(word)) {
20         return table.get(word);
21     }
22     return 0;
23 }

```

Note that a problem like this is actually relatively easy. Thus, the interviewer is going to be looking heavily at how careful you are. Did you check for error conditions?

16.3 Intersection: Given two straight line segments (represented as a start point and an end point), compute the point of intersection, if any.

pg 181

SOLUTION

We first need to think about what it means for two line segments to intersect.

For two infinite lines to intersect, they only have to have different slopes. If they have the same slope, then they must be the exact same line (same y-intercept). That is:

slope 1 != slope 2

OR

slope 1 == slope 2 AND intersect 1 == intersect 2

For two straight lines to intersect, the condition above must be true, *plus* the point of intersection must be within the ranges of each line segment.

extended infinite segments intersect

AND

intersection is within line segment 1 (x and y coordinates)

AND

intersection is within line segment 2 (x and y coordinates)

What if the two segments represent the same infinite line? In this case, we have to ensure that some portion of their segments overlap. If we order the line segments by their x locations (start is before end, point 1 is before point 2), then an intersection occurs only if:

Assume:

start1.x < start2.x && start1.x < end1.x && start2.x < end2.x

Then intersection occurs if:

start2 is between start1 and end1

We can now go ahead and implement this algorithm.

```

1 Point intersection(Point start1, Point end1, Point start2, Point end2) {
2     /* Rearranging these so that, in order of x values: start is before end and
3      * point 1 is before point 2. This will make some of the later logic simpler. */
4     if (start1.x > end1.x) swap(start1, end1);
5     if (start2.x > end2.x) swap(start2, end2);
6     if (start1.x > start2.x) {
7         swap(start1, start2);
8         swap(end1, end2);
9     }
10
11     /* Compute lines (including slope and y-intercept). */
12     Line line1 = new Line(start1, end1);
13     Line line2 = new Line(start2, end2);
14
15     /* If the lines are parallel, they intercept only if they have the same y
16      * intercept and start 2 is on line 1. */
17     if (line1.slope == line2.slope) {
18         if (line1.yintercept == line2.yintercept &&
19             isBetween(start1, start2, end1)) {
20             return start2;
21         }
22         return null;
23     }
24
25     /* Get intersection coordinate. */
26     double x = (line2.yintercept - line1.yintercept) / (line1.slope - line2.slope);
27     double y = x * line1.slope + line1.yintercept;
28     Point intersection = new Point(x, y);
29
30     /* Check if within line segment range. */
31     if (isBetween(start1, intersection, end1) &&
32         isBetween(start2, intersection, end2)) {
33         return intersection;
34     }
35     return null;
36 }
37
38 /* Checks if middle is between start and end. */
39 boolean isBetween(double start, double middle, double end) {
40     if (start > end) {
41         return end <= middle && middle <= start;
42     } else {
43         return start <= middle && middle <= end;
44     }
45 }
46
47 /* Checks if middle is between start and end. */
48 boolean isBetween(Point start, Point middle, Point end) {
49     return isBetween(start.x, middle.x, end.x) &&
50         isBetween(start.y, middle.y, end.y);
51 }
52
53 /* Swap coordinates of point one and two. */
54 void swap(Point one, Point two) {
55     double x = one.x;
56     double y = one.y;

```

```
57     one.setLocation(two.x, two.y);
58     two.setLocation(x, y);
59 }
60
61 public class Line {
62     public double slope, yintercept;
63
64     public Line(Point start, Point end) {
65         double deltaY = end.y - start.y;
66         double deltaX = end.x - start.x;
67         slope = deltaY / deltaX; // Will be Infinity (not exception) when deltaX = 0
68         yintercept = end.y - slope * end.x;
69     }
70
71     public class Point {
72         public double x, y;
73         public Point(double x, double y) {
74             this.x = x;
75             this.y = y;
76         }
77
78         public void setLocation(double x, double y) {
79             this.x = x;
80             this.y = y;
81         }
82     }
```

For simplicity and compactness (it really makes the code easier to read), we've chosen to make the variables within `Point` and `Line` public. You can discuss with your interviewer the advantages and disadvantages of this choice.

16.4 Tic Tac Win: Design an algorithm to figure out if someone has won a game of tic-tac-toe.

pg 181

SOLUTION

At first glance, this problem seems really straightforward. We're just checking a tic-tac-toe board; how hard could it be? It turns out that the problem is a bit more complex, and there is no single "perfect" answer. The optimal solution depends on your preferences.

There are a few major design decisions to consider:

1. Will `hasWon` be called just once or many times (for instance, as part of a tic-tac-toe website)? If the latter is the case, we may want to add pre-processing time to optimize the runtime of `hasWon`.
2. Do we know the last move that was made?
3. Tic-tac-toe is usually on a 3x3 board. Do we want to design for just that, or do we want to implement it as an $N \times N$ solution?
4. In general, how much do we prioritize compactness of code versus speed of execution vs. clarity of code? Remember: The most efficient code may not always be the best. Your ability to understand and maintain the code matters, too.

Solution #1: If hasWon is called many times

There are only 3^9 , or about 20,000, tic-tac-toe boards (assuming a 3x3 board). Therefore, we can represent our tic-tac-toe board as an `int`, with each digit representing a piece (0 means Empty, 1 means Red, 2 means Blue). We set up a hash table or array in advance with all possible boards as keys and the value indicating who has won. Our function then is simply this:

```
1 Piece hasWon(int board) {
2     return winnerHashtable[board];
3 }
```

To convert a board (represented by a char array) to an `int`, we can use what is essentially a “base 3” representation. Each board is represented as $3^0v_0 + 3^1v_1 + 3^2v_2 + \dots + 3^8v_8$, where v_i is a 0 if the space is empty, a 1 if it's a “blue spot” and a 2 if it's a “red spot.”

```
1 enum Piece { Empty, Red, Blue };
2
3 int convertBoardToInt(Piece[][] board) {
4     int sum = 0;
5     for (int i = 0; i < board.length; i++) {
6         for (int j = 0; j < board[i].length; j++) {
7             /* Each value in enum has an integer associated with it. We
8              * can just use that. */
9             int value = board[i][j].ordinal();
10            sum = sum * 3 + value;
11        }
12    }
13    return sum;
14 }
```

Now looking up the winner of a board is just a matter of looking it up in a hash table.

Of course, if we need to convert a board into this format every time we want to check for a winner, we haven't saved ourselves any time compared with the other solutions. But, if we can store the board this way from the very beginning, then the lookup process will be very efficient.

Solution #2: If we know the last move

If we know the very last move that was made (and we've been checking for a winner up until now), then we only need to check the row, column, and diagonal that overlaps with this position.

```
1 Piece hasWon(Piece[][] board, int row, int column) {
2     if (board.length != board[0].length) return Piece.Empty;
3
4     Piece piece = board[row][column];
5
6     if (piece == Piece.Empty) return Piece.Empty;
7
8     if (hasWonRow(board, row) || hasWonColumn(board, column)) {
9         return piece;
10    }
11
12    if (row == column && hasWonDiagonal(board, 1)) {
13        return piece;
14    }
15
16    if (row == (board.length - column - 1) && hasWonDiagonal(board, -1)) {
17        return piece;
18    }
19 }
```

```

19
20     return Piece.Empty;
21 }
22
23 boolean hasWonRow(Piece[][] board, int row) {
24     for (int c = 1; c < board[row].length; c++) {
25         if (board[row][c] != board[row][0]) {
26             return false;
27         }
28     }
29     return true;
30 }
31
32 boolean hasWonColumn(Piece[][] board, int column) {
33     for (int r = 1; r < board.length; r++) {
34         if (board[r][column] != board[0][column]) {
35             return false;
36         }
37     }
38     return true;
39 }
40
41 boolean hasWonDiagonal(Piece[][] board, int direction) {
42     int row = 0;
43     int column = direction == 1 ? 0 : board.length - 1;
44     Piece first = board[0][column];
45     for (int i = 0; i < board.length; i++) {
46         if (board[row][column] != first) {
47             return false;
48         }
49         row += 1;
50         column += direction;
51     }
52     return true;
53 }

```

There is actually a way to clean up this code to remove some of the duplicated code. We'll see this approach in a later function.

Solution #3: Designing for just a 3x3 board

If we really only want to implement a solution for a 3x3 board, the code is relatively short and simple. The only complex part is trying to be clean and organized, without writing too much duplicated code.

The code below checks each row, column, and diagonal to see if there is a winner.

```

1  Piece hasWon(Piece[][] board) {
2      for (int i = 0; i < board.length; i++) {
3          /* Check Rows */
4          if (hasWinner(board[i][0], board[i][1], board[i][2])) {
5              return board[i][0];
6          }
7
8          /* Check Columns */
9          if (hasWinner(board[0][i], board[1][i], board[2][i])) {
10             return board[0][i];
11         }

```

```

12     }
13
14     /* Check Diagonal */
15     if (hasWinner(board[0][0], board[1][1], board[2][2])) {
16         return board[0][0];
17     }
18
19     if (hasWinner(board[0][2], board[1][1], board[2][0])) {
20         return board[0][2];
21     }
22
23     return Piece.Empty;
24 }
25
26 boolean hasWinner(Piece p1, Piece p2, Piece p3) {
27     if (p1 == Piece.Empty) {
28         return false;
29     }
30     return p1 == p2 && p2 == p3;
31 }

```

This is an okay solution in that it's relatively easy to understand what is going on. The problem is that the values are hard coded. It's easy to accidentally type the wrong indices.

Additionally, it won't be easy to scale this to an NxN board.

Solution #4: Designing for an NxN board

There are a number of ways to implement this on an NxN board.

Nested For-Loops

The most obvious way is through a series of nested for-loops.

```

1 Piece hasWon(Piece[][] board) {
2     int size = board.length;
3     if (board[0].length != size) return Piece.Empty;
4     Piece first;
5
6     /* Check rows. */
7     for (int i = 0; i < size; i++) {
8         first = board[i][0];
9         if (first == Piece.Empty) continue;
10        for (int j = 1; j < size; j++) {
11            if (board[i][j] != first) {
12                break;
13            } else if (j == size - 1) { // Last element
14                return first;
15            }
16        }
17    }
18
19    /* Check columns. */
20    for (int i = 0; i < size; i++) {
21        first = board[0][i];
22        if (first == Piece.Empty) continue;
23        for (int j = 1; j < size; j++) {
24            if (board[j][i] != first) {

```